

Proyecto Final Programación

Juego de Parques (Parchís) y POO

Integrantes:

Maria Paula Cardenas

Alexander Cortes

Miguel Morales

Sofia Santos

Docente:

Alexander Parrado

Resumen

El propósito de este proyecto fue la realización de un código en el cual se demuestra la implementación de la lógica de programación y la adición de una interfaz gráfica, en el cual se permite el desarrollo de los temas previamente suministrados permitiendo el avance de este para culminar con un juego funcional, en el cual se puede jugar desde dos jugadores hasta cuatro.

Como resultado principal, se destaca la capacidad para la creación y modificación de funciones, clases y el desarrollo de interfaz gráfica mediante PyQt 6. Con el fin de generar un código con la capacidad de mantener un proceso de juego con dos a cuatro jugadores activos en tiempo real, manteniendo una experiencia de recreación y competitividad realista hasta obtener un ganador.

Palabras clave: Python , parques , apartado gráfico

Introducción

Con el fin de evidenciar los conocimientos previos y adquiridos durante el desarrollo de la asignatura de programación, se impuso el reto de modificar una serie de códigos previamente asignados, para así poder crear y explicar la lógica y la interfaz gráfica de un juego de parques creados mediante la herramienta PyQt 6.

Mediante la implementación de estas herramientas, se pudo desarrollar el juego teniendo la ventaja de poder interactuar con el código para la modificación de éste, permitiendo así el control de cada parámetro y con ello llevar a la culminación del proyecto con la presentación del juego funcional en el que se esperaba una relación indirecta de la parte lógica como cerebro del código y con la interfaz gráfica como el cuerpo o representación de este para que el usuario pueda desarrollar la partida de una manera clara y constructiva en la cual se podrán efectuar las diversas acciones con las que se desarrolla la partida como realizar reroll a los dados y con el valor dado por estos se podrá liberar las fichas y desplazarlas por el tablero.

Requerimientos

Los requerimientos presentados para la realización de un código en el cual se efectúa una partida completa de parqués, se fueron evidenciando mediante el avance en la misma programación y dictados por los programadores. Estos requerimientos se centran en lo que debe hacer el código, iniciando principalmente enfocados a la funcionalidad neta del programa como son la lógica principal del juego, las reglas predeterminadas o variantes, acciones y movimientos dentro del tablero e información del servidor. En la presente lista se especifican cada uno de ellos:

- Permitir un juego simultáneo de dos (2) jugadores.
- Lanzamiento de dados.
- Asignar automáticamente un color para cada jugador según el orden de llegada.
- Orden de inicio por lanzamiento de dados.
- Movimiento de fichas por el valor de cada dado.
- Salida de la cárcel por regla establecida.
- Movimientos básicos en las casillas.
- Detección de capturas, casillas seguras y victoria de jugador.
- Repetición de turno con dobles.
- Sincronización entre jugadores.
- Permite la interacción mediante botones en la IU.

Posterior a lo referente sobre la lógica pura del juego se tomó en cuenta un aspecto relacionado a la calidad del sistema, estos requerimientos se basan en la experiencia de los jugadores al interactuar con la interfaz y en la lectura y adaptación del código.

- Interfaz comprensible para cualquier usuario.
- Permite juego simultáneo entre varios clientes.
- Estructura separada en:
 - Lógica del juego.
 - Comunicación cliente a servidor.
 - Interfaz gráfica.
- Actualización de estado a tiempo real.
- Sin bloqueos de interfaz.

Con requerimientos establecidos se procedió con la generación del código, en el cual según se fue avanzando se visualizaron algunos más para el desarrollo. En este caso, se requirió la definición de una estructura para el cliente y el servidor, también para almacenar datos como: jugadores, fichas, turnos, dados y estado del tablero. Dentro del desarrollo de la lógica se tomó la implementación de la lógica de los turnos, reglas específicas de movimientos de fichas, validación para estos mismos movimientos, dados dobles, capturas y la finalización del juego.

Procedimiento

El desarrollo de la práctica se dividió en seis etapas, cada una enfocada en una parte diferente del sistema.

1. Arquitectura general del sistema

El programa se organizó en siete archivos, cada uno con una tarea específica.

El flujo de trabajo es el siguiente: el jugador hace una acción en su pantalla, esa acción se envía al servidor, el servidor la procesa y devuelve el nuevo estado del juego a todos los jugadores conectados, quienes ven el tablero actualizado al instante.

2. Motor de juego (game_engine.py)

Este archivo contiene todas las reglas del Parqués. Guarda en todo momento la información de la partida: quién está jugando, dónde están las fichas, qué salió en los dados y si hay turno extra.

Cuando un jugador mueve una ficha, el programa calcula a dónde debe ir según la posición actual y el valor del dado. Distingue si la ficha está en el recorrido normal, si está a punto de entrar a su pasillo de color, o si ya está dentro de él. Si el resultado supera la meta, el movimiento no se permite.

Al lanzar los dados, el comportamiento cambia según el momento de la partida: al inicio sirve para decidir quién empieza, y durante el juego activa reglas adicionales si ambos dados muestran el mismo número.

3. Adaptador del servidor (server_adapter.py)

Este archivo actúa como un intermediario: recibe el mensaje del jugador, identifica qué acción quiere realizar y llama a la función correspondiente del motor de juego

Si llega un mensaje que no corresponde a ninguna acción conocida, el sistema responde con un aviso de error sin interrumpir el funcionamiento.

4. Conexión entre servidor y jugadores

El servidor mantiene un registro de los jugadores conectados y sabe a quién enviarle cada mensaje: a veces la respuesta va a todos los jugadores, otras veces solo al que hizo la acción. Todo esto ocurre de forma automática sin que el jugador lo note.

Por su parte, cada cliente mantiene la conexión activa en segundo plano, de modo que la pantalla del juego nunca se congela mientras espera respuesta del servidor.

5. Interfaz gráfica (interfaz.py)

La pantalla del juego muestra el tablero con las fichas de todos los jugadores. Cada vez que alguien hace un movimiento, las fichas se actualizan visualmente de inmediato. Cuando dos fichas ocupan la misma casilla, se muestran ligeramente separadas para que ambas sean visibles.

Los controles son simples: un botón para lanzar los dados, un selector para elegir la ficha y otro para elegir con qué dado moverla, y un botón para confirmar el movimiento. Al abrir el programa, se pide el nombre del jugador y la ventana se conecta sola al servidor.

6. Cliente de pruebas (test_client.py)

Antes de conectar la interfaz gráfica, se usó un cliente de consola para probar cada acción del servidor por separado. Esto permitió detectar errores en la lógica del juego de forma más rápida y sencilla, sin depender de los botones ni del tablero visual.

Resultados

Objetivo:

¿Cuál era el resultado esperado?

Se esperaba generar un código funcional que le permitiera al usuario tener una experiencia realista de un juego de parques virtual.

¿Cuál fue el resultado real?

Teniendo en cuenta los códigos suministrados, se implementaron funciones que facilitarían la interacción del cliente con la interfaz gráfica creada, logrando así el objetivo principal de crear un programa funcional y realista en el cual el usuario pueda interactuar de manera sencilla con el juego, disfrutando al máximo la experiencia.

Acciones:

¿Qué acciones específicas contribuyeron a cumplir con los resultados esperados?

Trabajo constante, entendimiento del proceso de programación y tener objetivos claros con un orden específico en el procedimiento.

¿Qué acciones específicas detraen para alcanzar los resultados esperados?

Los principales desafíos que afectaron al desarrollo del programa fueron el proceso de aprendizaje de PyQt 6, el mapeo preciso de las coordenadas del campo de juego, la aplicación sin errores de reglas de simulación complejas y las pruebas lúdicas con múltiples jugadores.

Objetivo:

¿Cuál es el resultado esperado a futuro?

Mejorar el movimiento de las fichas para que se puedan mover desde la casilla que le corresponda a cada una y no desde la casilla número uno (1) como está ocurriendo en este momento.

Estrategia:

¿Qué acciones incrementarán la probabilidad de cumplir con los resultados esperados futuros?

El soporte y ayuda proporcionada por el docente y la inteligencia artificial como soporte de ayuda para la creación de diversas acciones que debe realizar el código permitiendo un mejor desarrollo de las actividades

5. REFERENCIAS (si aplica)

OpenAI.(2026). ChatGPT. <https://chat.openai.com/>

López-Parrado, A. (s/fa). Entrega1-proyecto-1-2026.
<https://github.com/parrado/entrega1-proyecto-1-2026>

López-Parrado, A. (s/fb). conferencia-2/pyqt en main · parrado/p-source-code-I-2026.
<https://github.com/parrado/p-source-code-I-2026/tree/main/lecture-2/pyqt>

DeepSeek. (2026). Asistencia de programación. <https://deepseek.com/>

6. ANEXOS (opcional)

Nombre del Archivo	Descripción
game_engine.py	Motor de juego con reglas del parques
server_adapter.py	Adaptador del servidor
client_transport.py	Cliente independiente
interfaz.py	Interfaz gráfica principal con PyQt 6
test_client.py	Cliente de pruebas en consola

Enlace en donde se encuentran los archivos.py:

<https://github.com/sofia-santos19/C-digos-del-proyecto-de-programaci-n>